实验 3 软件架构与界面设计报告

项目：AI 需求歧义审查器

学习通提交说明

- **提交平台：** 只向学习通对应的“实验报告”作业上传一份 PDF 文件。
- **文件命名：** 班级-学号-姓名-实验 3-软件架构与界面设计.pdf。
- **PDF 内容：** 需求与设计对应表、技术选型与架构决策、软件与外部环境关系图、总体架构图与模块说明、数据与接口设计、核心流程图、关键界面原型、设计检查记录及仓库版本证据。
- **截止时间：** 以学习通作业设置为准。

1. 实验目的

- 把实验 2 的需求说明转化为可指导后续开发的软件架构、模块边界、数据模型、接口和交互方案。
- 形成与个人能力匹配的技术选型，并保留关键决策依据。
- 完成关键设计，检查需求、架构与界面之间是否一致，为实验 4 的软件基础功能开发提供设计基线。

2. 实验内容

2.1 需求输入与设计范围

- 从实验 2 中选定一条核心用户流程，列出该流程对应的用户故事、功能需求、非功能需求和验收条件，作为本次设计的输入。
- 运用软件工程课程中学习的双向追踪原则，先建立“需求编号—准备完成的设计内容—对应图或文档”对应表，确保每项核心需求都有后续设计任务，后续也能从设计内容追溯到需求来源。
- 初步划分普通软件功能与 AI 功能的边界，说明哪些任务可以由确定的程序逻辑完成，哪些任务准备交给 LLM 处理，以及哪些结果需要程序校验或用户确认。本阶段不要求确定 RAG、Agent 等具体实现方法。

2.2 技术选型与架构决策

- 结合已学习的程序设计语言和软件设计约束，从本人熟悉程度、实现成本、框架生态、AI 接入、测试和运行环境等方面比较候选方案。比较可以使用简单表格，不要求罗列过多技术或原因。

- 对开发语言与框架、数据保存方式、软件体系结构风格等关键选择编写简短的设计决策说明。设计决策说明也称架构决策记录（ADR），只需写清要解决的问题、考虑过的方案、最终选择、理由和可能的不足。
- 为一项可能影响项目继续开发的关键依赖准备替代方案，例如模型服务、数据库或主要框架。

2.3 软件架构与模块设计

- 绘制软件与外部环境关系图（也称简化的系统上下文图），标出用户、个人软件、模型或模型服务以及项目需要访问的外部系统或数据源，说明哪些内容属于本软件、哪些属于外部依赖。模型应根据实际运行和调用方式放在合适位置，并注明使用本地模型还是云端模型服务。图形可以参考已经学习的数据流图，用简单的方框和带名称的箭头表示。
- 绘制软件总体架构图和模块结构图，根据项目特点选择已学习的分层、MVC 等体系结构风格，并可使用包图或构件图表示界面、业务、数据和 AI 能力等模块。两张图可以合并，但要写明各模块的职责和依赖关系。
- 在总体架构图中标明主要数据、调用方向或交互关系，并制作“用户—普通程序—模型—工具职责划分表”，说明核心流程中每一方负责什么以及由谁检查 AI 结果。

2.4 数据、接口与核心流程设计

- 使用已经学习的类图、ER 图或其他合适的图形描述核心数据，说明主要数据对象、关键字段、约束和关系，并标明需要保存的数据和敏感数据。
- 设计支撑核心用户流程的主要接口，说明接口由哪个模块提供、由谁调用、用途、输入、输出和失败情况，并至少给出一个正常调用与错误调用示例。
- 选择已经学习的顺序图、活动图、状态图或普通流程图，描述一次核心任务从用户输入到结果反馈的过程，包含正常路径、失败处理以及必要的人工确认点。

2.5 界面原型与交互设计

- 绘制页面结构图、导航关系图或顺序图，说明用户如何从软件入口进入核心功能，以及如何在页面之间跳转、返回、退出或重新操作。
- 完成不少于 3 张关键界面原型图，至少包括首页或入口页、核心任务页和结果页。界面原型是编码前的页面草图，用于确定页面上显示什么以及怎样操作，不要求编写前端代码，可以使用 Figma、墨刀、Axure 等工具，也可以使用清晰的**手绘图**。
- 结合已经学习的静态元素、动态元素、用户输入元素和用户命令元素，在原型中标注主要信息、输入区域、按钮、输入限制和操作反馈，并补充与项目有关的加载、空数据、错误、重新操作或人工确认状态。

2.6 设计一致性检查与版本保存

- 完善 2.1 建立的需求与设计对应表，补充“架构模块—数据或接口—界面—验收条件”等内容，逐项检查实验 2 中的核心需求是否已经落实，不需要重新制作一张内容重复的表格。
- 记录检查范围、发现的问题和设计调整；如果未发现问题，应说明检查依据。将架构图、数据模型图、流程图、界面原型图和其他设计文件保存到个人 GitHub/Gitee 仓库并提交。

3. 操作记录

以下记录围绕实验 2 确定的核心流程“提交需求文本并获得结构化歧义审查报告”展开。图中方框表示本软件或外部依赖，箭头表示数据或调用方向。

3.1 需求与设计对应表

需求编号	核心需求/验收条件	设计内容	对应图或文档
R1	用户可粘贴需求文本并启动审查；空文本应提示错误	Review 页面输入区、非空校验、POST /api/reviews	图 4、接口表
R2	支持 DOCX/文字型 PDF 上传并回填文本；非法格式或超限应失败	DocumentService 解析、上传接口、前端上传状态	图 1、图 4
R3	报告应包含原文引用、风险等级、问题类型、待确认问题和验收标准	AiReviewService 结构化 JSON、结果页筛选与人工确认	图 2、图 3、图 5
R4	用户可追问并保留历史版本	follow-up 接口、Review.versionsJson、结果页追问区	接口表、数据模型说明
R5	结果可按问题类型筛选，便于定位返工风险	前端 visibleIssues 过滤、结果页问题卡片	图 5
NFR1	API Key 不暴露给浏览器，模型异常需可理解地反馈	模型调用仅在后端；异常统一转为 error 响应	架构决策 ADR-03
NFR2	本地演示无需外部模型服务	AI_MODE=mock，保留 Qwen 适配替代路径	ADR-04

3.2 技术选型与架构决策

决策项	候选方案	最终选择	理由与不足
前端	Vue 3 / React	Vue 3 + Vite + Element Plus	已有项目基础、组件丰富、适合快速原型；不足是依赖生态版本需维护
后端	Spring Boot / Node.js	Spring Boot 3 + Java 17	熟悉 Java，REST、文件上传和异常处理清晰；不足是启动和依赖体积较大
数据	H2 / SQLite / 云数据库	H2 文件数据库	零配置、适合课程演示和历

			史版本；不足是单机并发与持久化能力有限
AI 接入	仅 Mock / 直接前端调用 / 后端适配	后端 AiReviewService 适配 Mock 与千问	保护 API Key，便于替换模型；不足是模型输出仍需加强约束
架构风格	单体脚本 / 分层 MVC	分层 + MVC	界面、接口、业务、AI、数据边界清楚；不足是当前规模下存在少量样板代码

ADR-01：选择 Vue 3 + Spring Boot 分层架构，解决课程原型需要快速迭代且保持模块边界的问题。ADR-02：选择 H2 文件数据库，降低部署成本。ADR-03：模型调用放在后端，避免密钥泄露。ADR-04：模型服务不可用时使用 Mock 报告作为替代方案，保证核心流程仍可演示。

3.3 软件架构与模块设计

本节对应实验内容 2.3，采用软件与外部环境关系图、总体架构图和模块结构图逐层说明系统边界、分层职责和模块依赖。

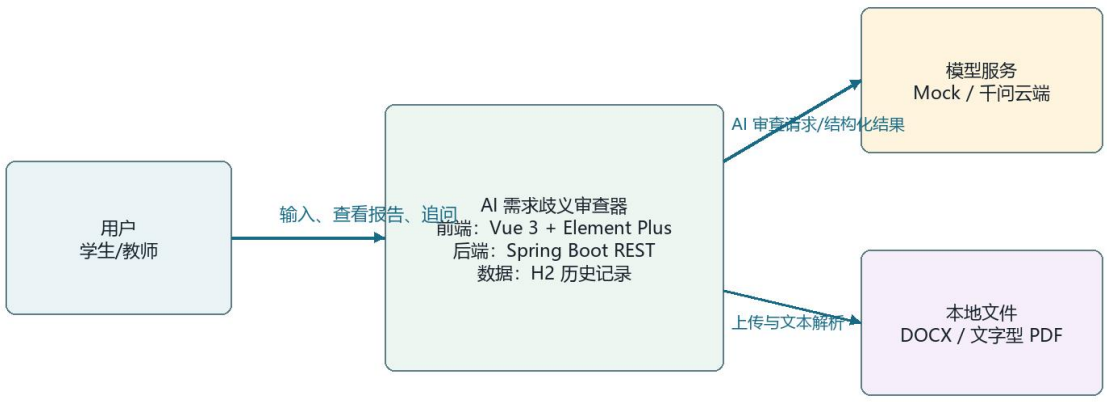


图 1 软件与外部环境关系图

用户、前端与后端属于本软件；Mock/Qwen 模型服务、DOCX/PDF 文件和 H2 运行文件属于外部依赖或运行环境。当前默认使用 Mock，接入千问时由后端访问云端模型服务。

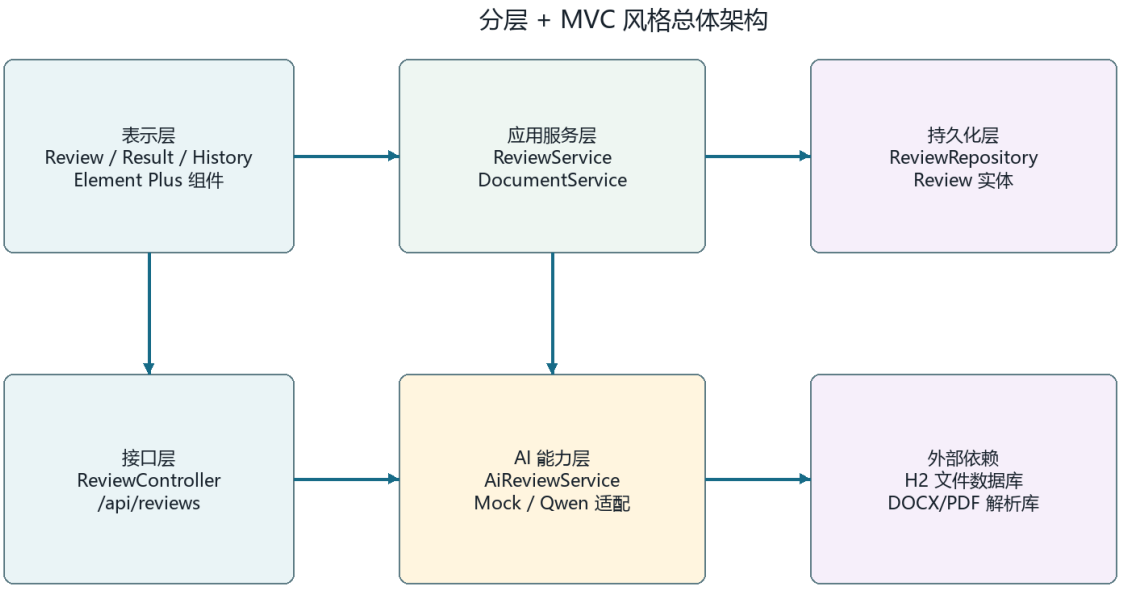


图 2 总体架构图

总体架构图与模块结构图的对应关系：表示层对应审查工作区、结果与追问、历史记录三个前端模块；接口层对应 ReviewController；应用服务层对应 ReviewService；AI 能力层对应 AiReviewService，文档解析由 DocumentService 负责；持久化层对应 Review、ReviewRepository 和 H2。模块之间按“页面模块→接口模块→业务/能力模块→数据模块”方向调用，外部模型服务和 DOCX/PDF 解析库只通过 AI 或文档模块进入系统。

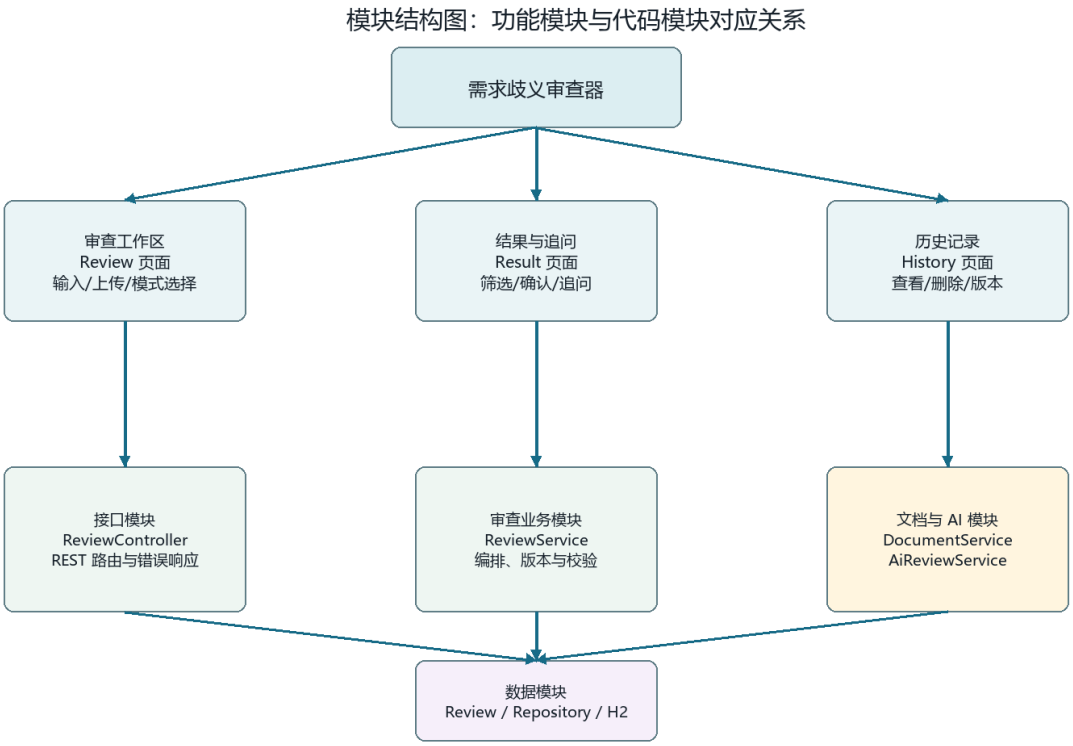


图 3 模块结构图

模块	**主要职责**	**依赖模块**
审查工作区	输入需求、上传文件、选择审查模式、展示加载和错误状态	接口模块
结果与追问	展示风险与问题、按类型筛选、提交追问、人工确认	接口模块
历史记录	查询、打开和删除历史审查记录	接口模块
接口模块	提供 REST 路由、参数接收和统一错误响应	审查业务模块、文档与 AI 模块
审查业务模块	编排审查、保存当前报告、维护追问版本	文档与 AI 模块、数据模块
文档与 AI 模块	解析 DOCX/PDF，调用 Mock 或 Qwen 并生成结构化结果	外部文件、模型服务
数据模块	保存 Review、报告 JSON、版本 JSON 和创建时间	H2 文件数据库
参与方	**主要职责**	**结果检查者**
用户	提供需求、选择模式、查看报告、确认待补条件、追问或重新提交	用户最终确认需求是否可接受
普通程序	文本解析、格式/空值校验、接口编排、历史保存、筛选展示	后端校验字段和请求错误
模型	识别模糊表达、边界缺口、冲突、权限和可测试性问题，生成结构化建议	程序做 JSON 字段校验，用户做业务确认
工具/外部依赖	DOCX/PDF 解析、H2 持久化、云端	程序检查解析异常、超时和服务错误

	Qwen（可选）	

3.4 数据、接口与核心流程设计

核心数据对象为 Review：id（UUID，主键）、text（原始需求文本）、mode（审查模式）、reportJson（当前结构化报告）、versionsJson（追问前版本数组）、createdAt（创建时间）。报告内部包含 summary、riskLevel、counts 和 issues；issues 包含引用、位置、解释、缺失条件、待确认问题、验收标准、相关角色及返工风险。原始需求可能包含课程项目内容，不应上传敏感个人信息；当前版本未实现登录和多用户隔离。

接口	调用者/提供者	输入	输出	失败情况
POST /api/reviews	前端 / ReviewController	text、mode	ReviewResponse（报告与版本）	空文本、AI 异常、保存失败
POST /api/reviews/upload	前端 / ReviewController + DocumentService	multipart file（DOCX/PDF）	fileName、text、characters	格式不支持、解析失败、文件超限
POST /api/reviews/{id}/follow-up	结果页 / ReviewService	question	更新后的 ReviewResponse	记录不存在、追问为空、模型异常
GET /api/reviews、GET /api/reviews/{id}	历史页 / ReviewService	id（详情接口）	历史摘要或完整报告	记录不存在
DELETE /api/reviews/{id}	历史页 / ReviewController	id	204 No Content	记录不存在或删除失败

正常调用示例：POST /api/reviews，JSON 为 {"text":"用户可以尽快完成注册","mode":"全面审查"}，返回 200 和包含 issues 的结构化报告。错误调用示例：POST /api/reviews，JSON 为 {"text":" ","mode":"全面审查"}，前端阻止提交并显示“请先输入或上传需求文本”；若服务端异常，则返回 400 与 {"error":"..."}。

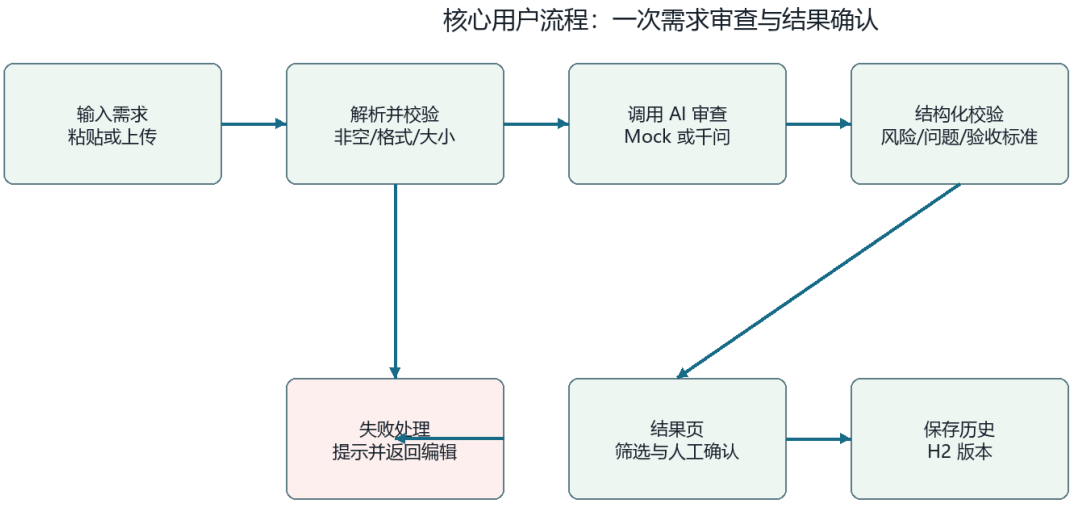


图 4 核心流程图

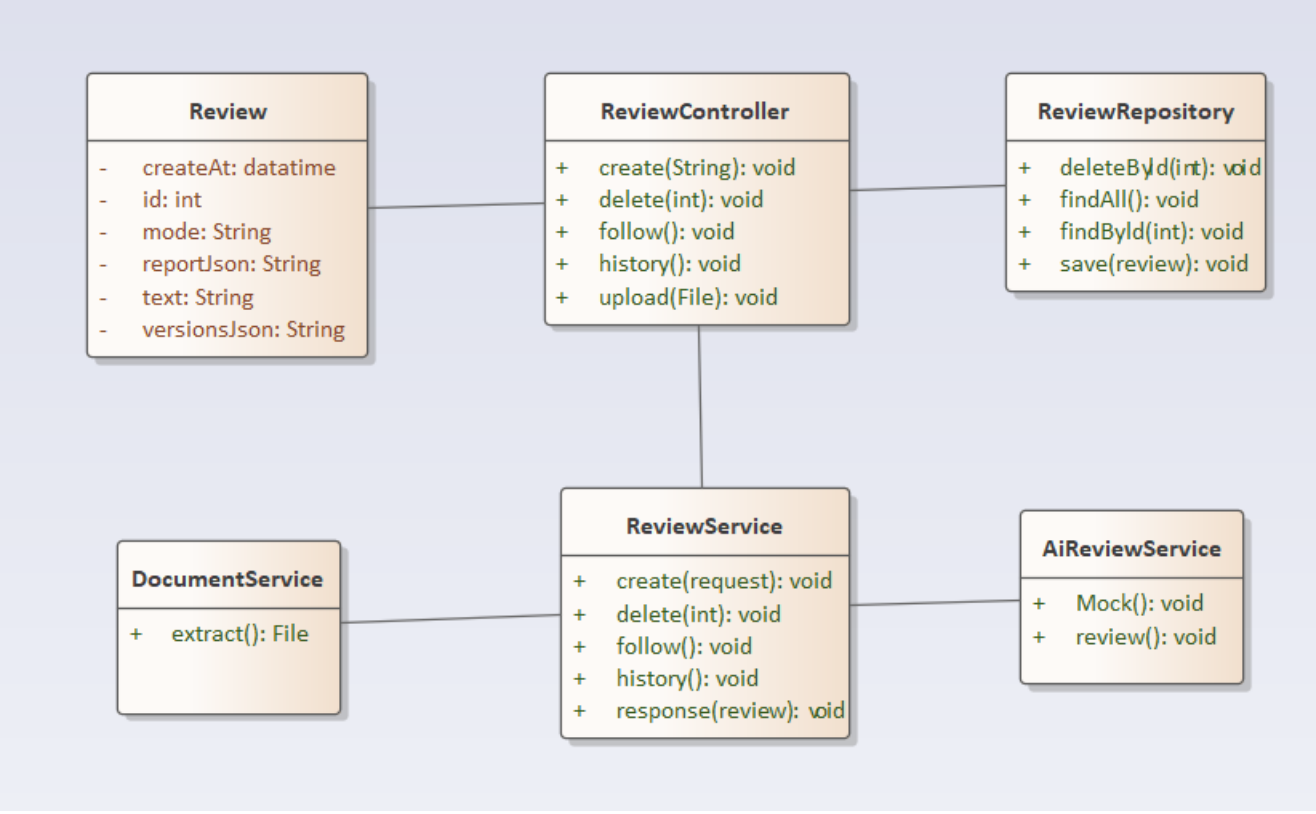


图 5 核心类图

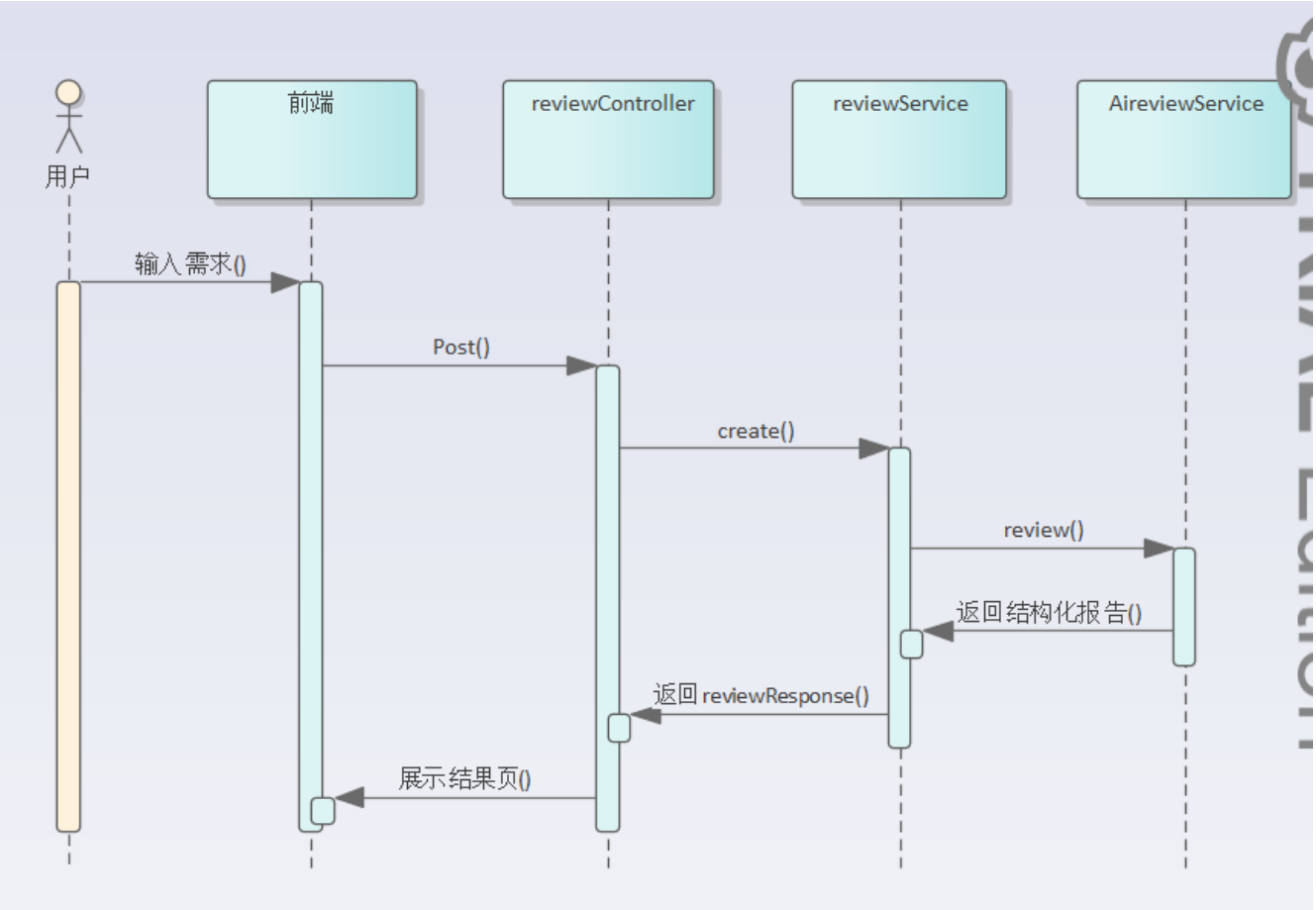


图 6 核心顺序图

类图与顺序图说明：Review 是核心持久化实体；ReviewController 接收前端请求并调用 ReviewService；ReviewService 编排 AI 审查、版本保存和历史查询；AiReviewService 负责 Mock/Qwen 适配与结构化校验；DocumentService 负责 DOCX/PDF 文本提取。顺序图展示一次正常审查请求，错误路径为解析失败或 AI 失败时由 Controller 返回 error，前端停留在编辑页。

正常路径为输入、解析校验、调用模型、结构化校验、结果展示、保存历史；解析失败或模型失败时回到编辑页并提示错误。结果页的“需要确认”与“可测试验收标准”是人工确认点，系统不把模型建议直接视为最终需求。

3.5 界面原型与交互设计

页面导航为：审查入口 → 结果页；结果页可返回编辑、修改后重新审查或追问；历史记录页可查看详情、删除记录并返回审查入口。项目现有界面已实现首页输入、结果报告和历史列表三类页面，以下截图作为关键界面原型/实现参考。



已有界面截图：首页/入口页

```
PS C:\Users\34856> cd "D:\deepseek harness workspace\AI_Request_check\frontend"
PS D:\deepseek harness workspace\AI_Request_check\frontend> npm run dev

> ambiguity-reviewer-web@0.1.0 dev
> vite


  VITE v5.4.21  ready in 6161 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

已有界面截图：核心任务页



图 7 首页/入口页原型



图 8 核心审查结果原型



图 9 结果页/历史记录交互原型

3.6 设计一致性检查与版本证据

本次检查覆盖需求追踪、架构模块、数据对象、接口、核心流程、界面原型和版本保存。检查结果表明：2.1 中的核心需求均能追溯到 3.3、3.4 和 3.5 的设计内容；架构模块、接口、数据和界面之间不存在明显矛盾。项目已提交到 GitHub，提交记录截图见图 10。

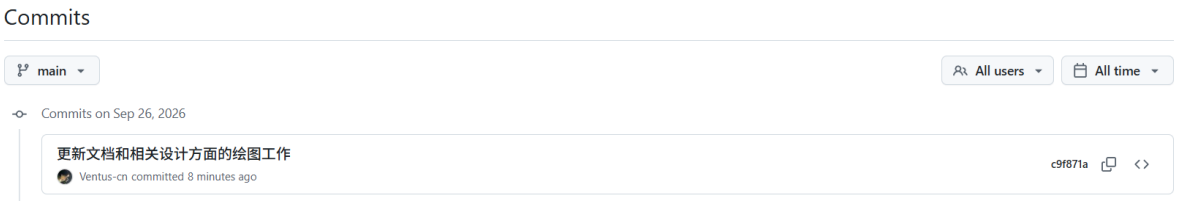


图 10 GitHub 提交记录（commit：c9f871a）

4. 实验小结

本实验将实验 2 中的“需求歧义审查”核心用户流程转化为可开发的设计基线。通过需求—设计对应表，明确了输入、解析、AI 审查、结果确认和历史追踪之间的追踪关系；通过分层架构，划分了前端展示、REST

接口、业务服务、AI 适配和 H2 持久化职责；通过数据与接口设计，明确了 Review、报告问题和追问版本的主要字段及失败处理；通过流程图和界面截图，补齐了从提交需求到查看结果的交互路径。

当前设计仍有边界：Mock 报告不能替代真实模型判断，扫描型 PDF 尚未支持 OCR，系统尚未提供登录、权限隔离、限流和文件安全扫描。后续开发应优先完成接口异常测试、模型输出严格校验、文件大小与内容安全检查，并在 GitHub/Gitee 提交图表和运行证据，作为实验 4 开发的基线。